

OSSP

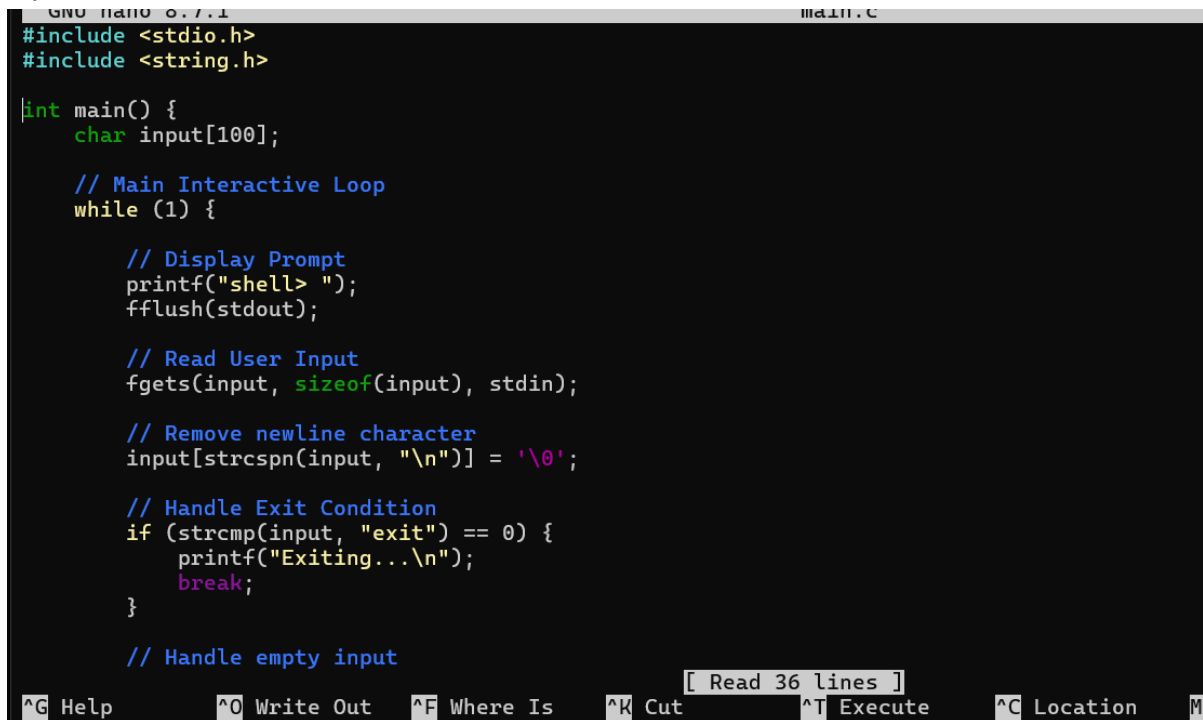
Skill Week-1

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task-1 : To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive

Input:-



```
GNU nano 2.9.3 main.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    // Main Interactive Loop
    while (1) {

        // Display Prompt
        printf("shell> ");
        fflush(stdout);

        // Read User Input
        fgets(input, sizeof(input), stdin);

        // Remove newline character
        input[strchr(input, '\n')] = '\0';

        // Handle Exit Condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting...\n");
            break;
        }

        // Handle empty input
    }
}
```

[Read 36 lines]

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M

```
nocturnal@nocturnal: /mnt/c, x + v
GNU nano 8.7.1 ma
    fflush(stdout);

    // Read User Input
    fgets(input, sizeof(input), stdin);

    // Remove newline character
    input[strcspn(input, "\n")] = '\0';

    // Handle Exit Condition
    if (strcmp(input, "exit") == 0) {
        printf("Exiting...\n");
        break;
    }

    // Handle empty input
    if (strlen(input) == 0) {
        continue;
    }

    // Process user input
    printf("You entered: %s\n", input);
}

return 0;
}
```

Output:-

```
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical$ ls
OSSP_2520090085
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical$ cd OSSP_2520090085
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085$ ls
Practical$
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085$ mkdir Skills
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085$ cd Skills
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ nano main.c
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ gcc main.c -o m
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ ./main
shell> Hello
You entered: Hello
shell> Shell
You entered: Shell
shell> Dell
You entered: Dell
shell> |
```

Loop To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

Input:-

```
nocturnal@nocturnal: /mnt/c, × + ▾
GNU nano 8.7.1 main1.c *
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    // Main Interactive Loop
    while (1) {

        // Display Prompt
        printf("shell> ");
        fflush(stdout);

        // Read User Input
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Handle Exit Condition
        if (strcmp(input, "exit") == 0 ||
            strcmp(input, "quit") == 0) {
            printf("Exiting interactive loop...\n");
            break;
        }
    }
}
```

```
nocturnal@nocturnal: /mnt/c, × + ▾
GNU nano 8.7.1 main1.c *
        if (fgets(input, sizeof(input), stdin) == NULL) {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Handle Exit Condition
        if (strcmp(input, "exit") == 0 ||
            strcmp(input, "quit") == 0) {
            printf("Exiting interactive loop...\n");
            break;
        }

        // Handle Empty Input
        if (strlen(input) == 0) {
            continue;
        }

        // Process Input
        printf("Command received: %s\n", input);
    }

    return 0;
}
```

Output:-

```

shell> exit
Exiting...
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ nano main.c
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ nano main1.c
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ gcc main1.c -o main1
nocturnal@nocturnal: /mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ ./main1
shell> Hello
Command received: Hello
shell> Lenovo
Command received: Lenovo
shell> L0Q
Command received: L0Q
shell>

```

Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

Input:-

```

GNU nano 8.7.1 main2.c *
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

struct termios original;

/* Restore terminal settings */
void disableRawMode() {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original);
}

/* Enable character-by-character input */
void enableRawMode() {
    tcgetattr(STDIN_FILENO, &original);
    atexit(disableRawMode);

    struct termios raw = original;
    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

```

```

GNU nano 8.7.1 main2.c *
}

/* Clear current input from screen */
void clearLine(int length) {
    for (int i = 0; i < length; i++)
        printf("\b \b");

    fflush(stdout);
}

int main() {
    char history[MAX_HISTORY][MAX_LENGTH];
    int historyCount = 0;

    enableRawMode();

    printf("Command History Program\n");
    printf("Use UP/DOWN arrows to recall commands.\n");
    printf("Type exit to quit.\n\n");

    while (1) {
        char input[MAX_LENGTH] = "";
        int length = 0;
        int historyIndex = historyCount;

        ^G Help      ^O Write Out   ^F Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo
        ^X Exit      ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^_ Go To Line   M-E Redo

```

```

char input[MAX_LENGTH] = "";
int length = 0;
int historyIndex = historyCount;

printf("shell> ");
fflush(stdout);

while (1) {

    char c;

    read(STDIN_FILENO, &c, 1);

    /* ENTER */
    if (c == '\n' || c == '\r') {
        input[length] = '\0';
        printf("\n");
        break;
    }

    /* BACKSPACE */
    if (c == 127) {
        if (length > 0) {
            length--;
            input[length] = '\0';

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-6 Copy

```

char input[MAX_LENGTH] = "";
int length = 0;
int historyIndex = historyCount;

printf("shell> ");
fflush(stdout);

while (1) {

    char c;

    read(STDIN_FILENO, &c, 1);

    /* ENTER */
    if (c == '\n' || c == '\r') {
        input[length] = '\0';
        printf("\n");
        break;
    }

    /* BACKSPACE */
    if (c == 127) {
        if (length > 0) {
            length--;
            input[length] = '\0';

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-6 Copy

```

char input[MAX_LENGTH] = "";
int length = 0;
int historyIndex = historyCount;

printf("shell> ");
fflush(stdout);

while (1) {

    char c;

    read(STDIN_FILENO, &c, 1);

    /* ENTER */
    if (c == '\n' || c == '\r') {
        input[length] = '\0';
        printf("\n");
        break;
    }

    /* BACKSPACE */
    if (c == 127) {
        if (length > 0) {
            length--;
            input[length] = '\0';

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-6 Copy

```
char input[MAX_LENGTH] = "";
int length = 0;
int historyIndex = historyCount;

printf("shell> ");
fflush(stdout);

while (1) {

    char c;

    read(STDIN_FILENO, &c, 1);

    /* ENTER */
    if (c == '\n' || c == '\r') {
        input[length] = '\0';
        printf("\n");
        break;
    }

    /* BACKSPACE */
    if (c == 127) {
        if (length > 0) {
            length--;
            input[length] = '\0';
        }
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/_ Go To Line M-E Redo M-G Copy

```
char input[MAX_LENGTH] = "";
int length = 0;
int historyIndex = historyCount;

printf("shell> ");
fflush(stdout);

while (1) {

    char c;

    read(STDIN_FILENO, &c, 1);

    /* ENTER */
    if (c == '\n' || c == '\r') {
        input[length] = '\0';
        printf("\n");
        break;
    }

    /* BACKSPACE */
    if (c == 127) {
        if (length > 0) {
            length--;
            input[length] = '\0';
        }
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/_ Go To Line M-E Redo M-G Copy

Output:-

```
nocturnal@nocturnal:~/mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ gcc main2.c -o main2
nocturnal@nocturnal:~/mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ ./main2
Command History Program
Use UP/DOWN arrows to recall commands.
Type exit to quit.

shell> hello
Command: hello
shell> PWD
Command: PWD
shell> PWD
```

Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

Input:-

```
Font Paragraph Styles
nocturnal@nocturnal: /mnt/c, × + ▾
GNU nano 8.7.1 main3.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

// Linked list node
typedef struct Node {
    char *data;
    struct Node *next;
} Node;

/* Add node to linked list */
void addNode(Node **head, const char *text) {
    Node *newNode = malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    newNode->data = malloc(strlen(text) + 1);

    if (newNode->data == NULL) {
        printf("Memory allocation failed!\n");
    }
}

[ Read 131 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-6 Copy
```

```
Font Paragraph Styles
nocturnal@nocturnal: /mnt/c, × + ▾
GNU nano 8.7.1 main3.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

// Linked list node
typedef struct Node {
    char *data;
    struct Node *next;
} Node;

/* Add node to linked list */
void addNode(Node **head, const char *text) {
    Node *newNode = malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    newNode->data = malloc(strlen(text) + 1);

    if (newNode->data == NULL) {
        printf("Memory allocation failed!\n");
    }
}

[ Read 131 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-6 Copy
```

```
GNU nano 8.7.1 main3.c
void freeList(Node *head) {
    Node *temp;

    while (head != NULL) {
        temp = head;
        head = head->next;

        free(temp->data);
        free(temp);
    }
}

int main() {
    /* Dynamic buffer allocation */
    size_t bufferSize = INITIAL_SIZE;
    char *buffer = malloc(bufferSize);

    if (buffer == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
}
```

```
nocturnal@nocturnal: /mnt/c, × + v
GNU nano 8.7.1 main3.c
}

printf("Enter a string: ");

/*
Read characters dynamically.
Resize buffer when required.
*/
size_t length = 0;
char ch;

while ((ch = getchar()) != '\n' && ch != EOF) {

    /* Prevent buffer overflow */
    if (length + 1 >= bufferSize) {

        bufferSize *= 2;

        char *temp = realloc(buffer, bufferSize);

        if (temp == NULL) {
            printf("Buffer resizing failed!\n");
            free(buffer);
            return 1;
        }
    }
}
```

```
nocturnal@nocturnal: /mnt/c, × + v
GNU nano 8.7.1 main3.c
}

printf("Enter a string: ");

/*
Read characters dynamically.
Resize buffer when required.
*/
size_t length = 0;
char ch;

while ((ch = getchar()) != '\n' && ch != EOF) {

    /* Prevent buffer overflow */
    if (length + 1 >= bufferSize) {

        bufferSize *= 2;

        char *temp = realloc(buffer, bufferSize);

        if (temp == NULL) {
            printf("Buffer resizing failed!\n");
            free(buffer);
            return 1;
        }
    }
}
```



```
GNU nano 8.7.1 main3.c

buffer[length] = '\0';

printf("\nStored string: %s\n", buffer);
printf("Final buffer size: %zu bytes\n", bufferSize);

/* Linked list */
Node *head = NULL;

addNode(&head, buffer);
addNode(&head, "Second Node");
addNode(&head, "Third Node");

displayList(head);

/* Release linked-list memory */
freeList(head);

/* Release dynamic buffer */
free(buffer);

printf("\nAll dynamically allocated memory released.\n");

return 0;
}
```

Output:-

```
shell> quit
Command: quit
shell> exit
Exiting...
nocturnal@nocturnal:/mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ nano main3.c
nocturnal@nocturnal:/mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ nano main3.c
nocturnal@nocturnal:/mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ gcc main3.c -o main3
nocturnal@nocturnal:/mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ ./main3
Enter a string: Sujit

Stored string: Sujit
Final buffer size: 10 bytes

Linked List:
Third Node
Second Node
Sujit

All dynamically allocated memory released.
nocturnal@nocturnal:/mnt/c/Users/vsbsu/Desktop/OSSP/Practical/OSSP_2520090085/Skills$ |
```